PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective 4
3. Product Purpose 5
4. System Functionalities 5
5. User Stories & Use Cases 8
6. Features Included (Scope Definition) 11
7. Features Not Included (Scope Control) 12
8. Assumptions & Constraints 12
9. Risks & Questions Throughout Development 13

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: DonateAI

Version: 1.0

1. Project Overview

- **Problem Statement:** Donation coordination in Malaysia is fragmented, especially during disaster situations. Donors are often willing to help but do not know which verified organizations in need require specific items, whether their donations are suitable, or whether their contributions reach the intended recipients. At the same time, organizations in need struggle to publish urgent needs efficiently, particularly when requests are communicated through informal channels such as WhatsApp messages.

  In addition, delivery coordination remains a major challenge. Donors may not have the time or ability to deliver items, while organizations in need lack structured logistics support to manage incoming donations. This results in delays, unfulfilled needs, and inefficient distribution of resources.

- **Target Domain:** AI-assisted donation allocation, disaster-response coordination, needs management for organizations in need, and delivery/logistics coordination.

- **Proposed Solution Summary:** DonateAI is a dual-mode AI-powered platform designed to improve coordination between donors and organizations in need.

  In normal mode, donors can browse live needs from verified organizations in need, donate physical items through an AI-assisted chatbot, upload item photos for screening, receive AI-based matching recommendations, and track delivery progress.

© UMHackathon 2026
Email: umhackathon@um.edu.my

In disaster mode, the system ingests disaster signals, activates emergency workflows, collects needs through outreach, converts responses into structured data using AI, and prioritizes urgent allocation of resources.

To address delivery challenges, DonateAI provides flexible logistics options, including self-delivery by donors and platform-supported delivery. Corporate partners act as logistics supporters or sponsors, helping reduce delivery costs and improve efficiency in resource distribution.

2. Background & Business Objective

- **Background of the Problem:** Traditional donation coordination relies heavily on manual communication and fragmented information sources. Donors often depend on social media posts, forwarded messages, or personal contacts to decide where to contribute. These sources are frequently outdated, duplicated, or unverified.

  During disaster situations, the problem becomes more severe. Organizations in need may urgently require essential items such as food, blankets, hygiene kits, or medical supplies, but these needs are often communicated through unstructured text messages. Coordinators must manually interpret, prioritize, and organize these requests while simultaneously managing logistics.

  Furthermore, delivery coordination is often overlooked. Without structured logistics support, donations may be delayed, misdirected, or never delivered.

.

- **Importance of Solving This Issue:** Solving this issue improves the efficiency, transparency, and fairness of donation coordination. A structured system ensures that donations are matched to real-time needs and delivered to verified organizations in need. In disaster scenarios, it enables faster response and prioritization of urgent requests. Additionally, introducing structured logistics options reduces the burden on donors and

© UMHackathon 2026

organizations in need, ensuring that delivery is reliable and efficient. Overall, this improves trust and accountability in the donation process.

- **Strategic Fit / Impact:** DonateAI demonstrates how artificial intelligence can be applied to solve real-world coordination problems by integrating intelligent matching, automation, and logistics support into a single platform. The system focuses on improving both allocation and delivery, ensuring that donations are not only matched correctly but also successfully delivered. This approach enhances overall system effectiveness and real-world impact.

3. Product Purpose

3.1. Main Goal of the System: The main goal of DonateAI is to provide an intelligent and structured donation coordination platform that connects donors, verified **organizations in need**, administrators, and logistics partners within a unified workflow. The system supports both everyday donation activities and disaster-response scenarios. In normal mode, it facilitates efficient donation matching and tracking. In disaster mode, it enables rapid coordination through disaster detection, outreach, structured need generation, route planning, and monitoring of resource distribution.

3.2. Intended Users (Target Audience):

- Everyday Individual Donors
- Organizations in need
- Corporate ESG Sponsors

4. System Functionalities

4.1. Description: DonateAI is an AI-assisted donation and disaster-response coordination platform that supports both everyday donation activities and emergency scenarios.

© UMHackathon 2026

The system enables users to browse needs, donate resources, manage requests from organizations in need, coordinate disaster operations, and track delivery outcomes. AI is primarily used for text-based processing, including donor interaction, donation matching, and extraction of structured information from unstructured inputs such as shelter messages.

The platform does not operate as a fully autonomous system. Instead, it follows a human-in-the-loop design, where AI assists in decision-making while administrators remain responsible for critical actions such as disaster activation, outreach, and publishing needs. This approach ensures both efficiency and reliability.

4.2. Key Functionalities:

- **Public Needs Browsing:** Allows users to view active needs from verified organizations in need without requiring login, with filtering based on category, urgency, and location.
- **AI Donation Assistant:** Provides a conversational interface that guides donors to input donation details such as item type, quantity, and delivery preference.
- **AI Allocation Planning:** Recommends suitable organizations in need by analyzing donor input and matching it with relevant and urgent needs.
- **Organization in Need Management:** Enables organizations to create, update, and manage their needs while tracking incoming donations and fulfillment progress.
- **Disaster Signal Handling:** Processes disaster signals from manual input and external sources, assigning confidence scores based on keywords, location, and severity indicators.
- **Disaster Mode Activation:** Allows administrators to activate crisis mode, enabling the system to prioritize disaster-related needs across the platform.
- **Shelter Outreach Communication:** Supports administrators in sending outreach messages to affected organizations in need to gather updated requirements.
- **AI-Assisted Need Extraction:** Converts unstructured shelter messages into structured data such as item category, quantity, and urgency, subject to administrator verification.

© UMHackathon 2026

- **Route Planning and Delivery Coordination:** Generates optimized delivery routes and stops based on current needs, allocations, and collection points.
- **Equity Monitoring:** Tracks resource distribution and identifies underserved organizations in need to ensure fair allocation.
- **Reporting:** Produces summary reports containing information on needs, allocations, deliveries, and overall system performance.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: DonateAI uses Z.AI (ILMU-GLM-5.1) as the primary model for conversational interaction, donation allocation planning, and structured data extraction from unstructured inputs. Fallback mechanisms are implemented to ensure system stability when AI responses are unavailable or invalid.

4.3.2. Prompting Strategy: The system adopts a **multi-step structured prompting strategy** to ensure consistent and reliable outputs. Each AI functionality, including chat-based donation intake, allocation planning, and need extraction, is handled using task-specific prompts. These prompts enforce structured JSON outputs to maintain compatibility with backend processing. The workflow is divided into multiple stages, starting from donation intake through conversational guidance, followed by allocation planning where donations are matched to suitable needs, and finally need extraction where unstructured shelter messages are converted into structured data. This structured approach improves control over AI responses, reduces the risk of hallucination, and ensures smoother integration with the system.

4.3.3. Context & Input Handling: The system maintains a localized chat history. If a user provides incomplete data (e.g., "I have clothes"), the AI enters a missing-fields loop to request the remaining constraints.

© UMHackathon 2026

4.3.4. Fallback & Failure Behavior: To ensure system reliability, several fallback mechanisms are implemented. When the AI produces invalid or incomplete outputs, the system applies rule-based logic as a backup to maintain functionality. In cases where the AI service is unavailable, the system returns a graceful error message to inform the user without causing disruption. If the extracted data contains uncertainty, it is flagged for admin review before being published to ensure accuracy. Additionally, critical actions such as disaster activation and publishing of needs always require human confirmation. These measures ensure that the system remains stable and usable even when AI components fail or produce unreliable results.

5. User Stories & Use Cases

5.1 User Story 1: Public Needs Browsing

- **User Stories:** As a public visitor, I want to browse active needs from verified organizations so that I can understand where help is currently needed before deciding to donate.
- **Use Cases (Main Interactions):** The public visitor opens the needs page without logging in. The system retrieves active needs from the database and displays them with organization information, category, quantity, urgency, and remaining amount. The visitor can filter needs by category or urgency and open a need detail page. If the visitor wants to donate, the system directs them to log in or create a donor account.

5.2 User Story 2: AI-Assisted Donor Donation

- **User Stories:** As a donor, I want to describe the items I want to donate through an AI assistant so that the system can recommend suitable organizations and needs.
- **Use Cases (Main Interactions):** The donor opens the donation assistant and enters item details such as category, quantity, condition, and location. The AI assistant asks follow-up questions if the information is incomplete. The donor may upload an item photo, and the system screens the image to check whether it matches the declared item and whether the

© UMHackathon 2026

condition is acceptable. The system then matches the donation to live needs and provides an allocation recommendation. After the donor confirms, the donation and allocation records are saved for tracking.

5.3 User Story 3: Admin Disaster Response

- **User Stories:** As an admin, I want to manage disaster events and coordinate responses so that aid is delivered efficiently.
- **Use Cases (Main Interactions):** The admin initiates disaster-related processes by ingesting signals manually or from external sources. The system scores and evaluates these signals, after which the admin activates a disaster event. Once activated, the system prioritizes disaster-related needs. The admin can trigger shelter outreach, collect responses, and use AI to extract structured needs. These needs are reviewed and published, making them available for donor matching.

5.4 User Story 4: Shelter Communication & Need Extraction

- **User Stories:** As an admin, I want to convert shelter messages into structured needs so that they can be used for donation matching.
- **Use Cases (Main Interactions):** Shelter responses are received through messaging services and stored in the system. The admin selects a message and triggers AI-based extraction. The system processes the unstructured message and suggests structured fields such as item type, quantity, and urgency. The admin reviews the output and confirms it before publishing it as an active need.

5.5 User Story 5: Receiver Need Management

- **User Stories:** As a receiver organization, I want to publish and manage needs so that donors can contribute the right items to my organization.
- **Use Cases (Main Interactions):** The receiver logs into the receiver portal and creates a new need by entering title, description, category, quantity, urgency, and optional image information. The system stores the need in the database and displays it on the public and

© UMHackathon 2026

donor needs pages when active. The receiver can later monitor incoming donations, review donation offers, and update fulfillment progress.

5.6 User Story 6: Donation Tracking

- **User Stories:** As a donor, I want to track the progress of my donation so that I know whether it has been allocated, delivered, and verified.
- **Use Cases (Main Interactions):** After confirming a donation, the donor opens the tracking page. The system retrieves the donation status, allocation records, and delivery progress. The donor can view each stage of the donation pipeline, including submission, allocation, pickup or drop-off, delivery, proof upload, and verification. If proof is available, the system displays the delivery confirmation status.

5.7 User Story 7: Route Planning

- **User Stories:** As an admin, I want to generate delivery routes from current donation allocations so that platform-supported deliveries can be coordinated efficiently.
- **Use Cases (Main Interactions):** The admin opens the operations or routes page and triggers route planning. The system retrieves active disaster needs, pending platform-delivery allocations, and available collection points. It creates a route plan with ordered route stops and expected quantities. The admin can then view the route, update stop status such as arrived or completed, and request proof when needed.

5.8 User Story 8: Equity Monitoring

- **User Stories:** As an admin, I want to monitor underserved shelters so that aid distribution remains fair during disaster response.
- **Use Cases (Main Interactions):** The admin triggers equity recomputation from the operations panel. The system compares requested quantities against fulfilled quantities across active disaster needs. It calculates fulfillment ratios and underserved scores. If a shelter or category is under-supported, the system creates an equity alert. The admin reviews these alerts and can adjust outreach, campaigns, or routing priorities.

© UMHackathon 2026

5.9 User Story 9: Corporate ESG Contribution Tracking

- **User Stories:** As a corporate ESG partner, I want to view contribution history and impact records so that my company can track measurable social impact.
- **Use Cases (Main Interactions):** The corporate partner logs into the corporate portal and views contribution records, donation history, and reports. The system displays supported causes, donation entries, and impact-related information. The corporate partner can use these records to support ESG reporting and internal impact documentation.

5.10 User Story 10: Admin Reporting

- **User Stories:** As an admin, I want to generate disaster response reports so that the team can review response performance and summarize outcomes.
- **Use Cases (Main Interactions):** The admin opens the reporting page and triggers report generation. The system collects current disaster data such as active needs, allocations, route plans, proof records, communications, and equity alerts. It creates a report run and displays the summary in the admin panel. The admin can use this report to evaluate response progress and prepare post-crisis documentation.

6. Features Included (Scope Definition)

- Public browsing of active needs without login
- Donor portal for submitting and tracking donations
- Organization in need portal for managing needs
- AI-powered donation assistant (chat-based guidance)
- AI-based donation matching and allocation planning
- Donation confirmation and tracking system
- Admin disaster management console
- Disaster signal ingestion and scoring
- Admin-controlled disaster activation

© UMHackathon 2026

- Shelter outreach and communication handling
- AI-assisted need extraction from messages
- Route planning and delivery coordination
- Equity monitoring and alert system
- Reporting and analytics generation

7. Features Not Included (Scope Control)

- Mobile application (native iOS/Android)
- Financial donation system
- Volunteer-based delivery marketplace
- Real-time chat between donors and organizations
- Fully automated disaster detection and activation
- Fully autonomous AI decision-making
- GPS-based delivery tracking
- Social media automation features

8. Assumptions & Constraints

- **LLM Cost Constraint:** Inference costs are minimized by isolating the AI call; the model is only pinged during the chat intake phase. Corporate bulk uploads bypass real-time chat and are matched asynchronously via a backend cron job to save tokens.
- **Technical Constraints:** Relies on the uptime of the api.ilmu.ai endpoint and Supabase free-tier limits.
- **Performance Constraints:** AI responses may introduce latency, especially under heavy load or large inputs.
- **User Input Constraint:** The system requires user or admin input for critical actions and cannot operate fully autonomously.

© UMHackathon 2026

Email: umhackathon@um.edu.my

9. Risks & Questions Throughout Development

- **Design Similarity:** The system reduces the risk of generating similar outputs across users by incorporating dynamic inputs such as user-provided item details, real-time needs data, and contextual variations in prompts. Since the AI operates on live data and user-specific inputs, the allocation results and responses are naturally differentiated. Additionally, structured prompting ensures that outputs are grounded in actual needs rather than generic patterns.

- **Frame Stability:** The system ensures stability by validating all structured outputs before rendering them in the user interface. Any incomplete or invalid AI responses are either corrected through fallback logic or flagged before being displayed. This prevents broken layouts or inconsistent data from appearing on the frontend. In cases where issues still occur, the system provides clear feedback to users and avoids committing unstable data into the database.

- **Drawing Interpretation**: Since the system processes unstructured inputs such as text or uploaded data, there is a risk that the AI may misinterpret unclear or ambiguous information. To mitigate this, the system applies validation checks and confidence scoring on AI outputs. If the extracted or interpreted data is uncertain, it is either refined through fallback logic or requires manual admin review before being published. This ensures that incorrect interpretations do not directly affect system operations or decision-making.

© UMHackathon 2026

Email: umhackathon@um.edu.my